

Document explicatif - BobApp

1. Étapes mises en œuvre dans les GitHub Actions

Flux d'Intégration Continue (CI) - [backend-ci.yml](#) & [frontend-ci.yml](#)

Ces workflows sont déclenchés automatiquement lors de chaque pull request ou d'un push sur la branche principale. Ils garantissent que le code ajouté ne casse pas l'application :

- Récupération du code (Checkout) : l'environnement télécharge le dernier code
- Configuration de l'environnement : mise en place de Java 11 pour le back-end et de Node.js pour Angular côté front-end
- Installation des dépendances : téléchargement des bibliothèques requises (via Maven pour le backend et npm pour le frontend)
- Exécution des tests et couverture : lancement des tests unitaires existants et génération automatique des rapports de couverture de code
- Analyse de qualité via SonarCloud/SonarQube : envoi des rapports générés vers Sonar pour analyser la qualité et la sécurité du code en continu

Flux de Déploiement Continu (CD) - [backend-docker-deploy.yml](#) & [frontend-docker-deploy.yml](#)

Une fois que les workflows CI sont validés (tests passés et analyse validée), ces actions prennent le relais pour préparer la livraison :

- Déclenchement conditionnel : s'exécute uniquement si l'étape de CI a réussi
- Connexion à Docker Hub : authentification au registre Docker via des secrets GitHub sécurisés
- Build et push des images Docker : création des images pour le frontend et le backend à partir de leurs Dockerfile respectifs
- Publication : les conteneurs prêts à l'emploi sont envoyés sur Docker Hub

2. Propositions de KPI (Indicateurs de qualité Sonar)

Pour s'assurer de la viabilité à long terme du projet, on utilise la "quality gate" par défaut de Sonar : Sonar Way. Voici les KPI proposés que tout nouveau code doit respecter pour que la pull request soit acceptée :

- Couverture de code : > 80% sur le nouveau code. Il est impératif que toute nouvelle fonctionnalité implémentée soit rigoureusement testée
- Bugs et vulnérabilités : 0 "New Blocker/Critical issues". Aucun bug ou faille critique de sécurité ne doit être introduit
- Dette technique: note A
- Code dupliqué : < 3% sur le nouveau code

3. Analyse des métriques et des avis utilisateurs

L'analyse des derniers commentaires révèle des problèmes majeurs :

- *“Impossible de poster une suggestion de blague, le bouton tourne et fait planter mon navigateur !”* = **Bug bloquant en production**
- *“j’ai remonté un bug sur le post de vidéo il y a deux semaines et il est encore présent !”* = **Cycle de résolution et de déploiement trop lent. L’absence d’automatisation créait un goulot d’étranglement fatal**
- *“Ça fait une semaine que je ne reçois plus rien / J’ai supprimé ce site”* = **Perte de confiance des utilisateurs due à un manque de communication et de réactivité**

Perspectives et actions futures

- Réactivité restaurée : Grâce à l'automatisation CI/CD, Bob n'a plus à déployer manuellement via FTP. Un bug corrigé sur le code peut maintenant être testé et déployé sur Docker Hub en quelques minutes
- Réduction des régressions : Le workflow oblige au passage des tests. Le navigateur ne plantera plus en production si des tests e2e ou d'intégration sont ajoutés pour valider ce comportement avant la mise en ligne
- Amélioration du suivi : La mise en place de la CI simplifie radicalement les contributions open source. Les développeurs externes verront rapidement si leur Pull Request est valide grâce aux indicateurs Sonar et aux actions GitHub